

Plano: camada de PLN entre o campo de busca e o TMDb

Projeto de PLN — busca e sugestão de filmes por texto livre (TMDb API)

Contexto

O backend (`backend/src/app/`) já expõe todos os endpoints TMDb necessários — `buscar_filmes` (título), `descobrir_filmes` (filtro por gênero/nota/período), `generos` (mapa id→nome) — e o frontend já tem um campo de busca funcional (`#campo-titulo` em `frontend/index.html` , tratado em `frontend/script.js`) que hoje só faz busca literal por título via `/busca` .

O objetivo agora é o passo que faltava: entender frases livres como *"queria um filme descontraído com boa avaliação que não seja muito antigo"* e traduzi-las nos parâmetros que `/descobrir` já aceita — **sem** adicionar nenhuma UI nova. O requisito explícito do usuário é que o campo de busca continue visualmente idêntico; a interpretação do texto acontece só no backend, de forma transparente para quem usa o site.

Decisão já tomada com o usuário: a extração será **100% Python puro** (regex, `unicodedata` , `dataclasses` , `stdlib`), sem spaCy/sentence-transformers/etc. Motivo: o backend está fixado em Python 3.14 (`backend/pyproject.toml`), e o spaCy + modelos `pt_core_news` atualmente não instalam nessa versão (trava por incompatibilidade do pydantic v1, bug reportado jan/2026); `sentence-transformers` / `torch` têm suporte incerto também. Isso elimina o risco de dependência quebrada antes da entrega e ainda é PLN de verdade (normalização, análise léxica, regras de negação, extração de expressões temporais) — só não usa biblioteca de terceiros.

Confirmado por exploração do repo: `backend/pyproject.toml` só lista `fastapi` , `pydantic` , `python-dotenv` , `requests` , `uvicorn[standard]` — nenhum framework de teste instalado (nem pytest).

A) Novo pacote `backend/src/app/services/pln/`

Espelha o padrão transporte-vs-domínio já usado em `services/tmdb/` , mas dividido por responsabilidade de PLN — cada módulo puro é testável isoladamente, sem rede:

arquivo	responsabilidade	I/O?
<code>normalizacao.py</code>	<code>normalizar(texto)</code> (lowercase + remove acento via <code>unicodedata</code> NFKD + colapsa espaços), <code>tokenizar(texto)</code> (regex <code>\w+</code>)	não
<code>lexico.py</code>	dicionários palavra→conceito: <code>GENEROS_POSITIVOS</code> (humor/adjetivo → nomes de gênero TMDb, ex. <code>"descontraído": ("Comedia", "Familia")</code>),	não

	<code>PALAVRAS_QUALIDADE_ALTA / _MUITO_ALTA ;</code> <code>buscar_generos_no_texto(tokens) ,</code> <code>buscar_qualidade_no_texto(texto) → (nota_minima,</code> <code>votos_minimos)</code>	
<code>periodo.py</code>	regex para "dos anos 90", "depois de 2015"/"antes de 2000", "recente"/"não muito antigo" (relativo a <code>date.today().year</code>), "antigo"/"clássico"; <code>extrair_periodo(texto) -></code> <code>PeriodoExtraido</code>	não
<code>negacao.py</code>	<code>separar_generos_negados(tokens, generos_no_texto) —</code> janela de ~3 tokens antes de cada gênero detectado procurando <code>nao / sem / nunca ; devolve (incluir, excluir)</code>	não
<code>generos_cache.py</code>	<code>mapa_id_para_nome() / mapa_nome_para_id() —</code> carrega <code>tmdb_filmes.generos()</code> uma vez (lazy, na 1ª pesquisa, não no boot) e cacheia em memória por processo, sem TTL; <code>invalidar_cache()</code> só para testes	sim — único módulo que fala com o TMDB
<code>extrator.py</code>	<code>FiltrosExtraidos</code> (dataclass) + <code>extrair_filtros(texto) -></code> <code>FiltrosExtraidos</code> , orquestra normalização → léxico → negação → período → resolve nomes de gênero em IDs via <code>generos_cache</code>	não (consome cache já carregado)
<code>pesquisa.py</code>	<code>pesquisar(texto, pagina, ano) -> ResultadoPesquisa —</code> decide fallback (item C) e chama <code>tmdb_filmes</code>	sim — via <code>tmdb_filmes</code>

`FiltrosExtraidos` (dataclass, não pydantic — consistente com o resto do domínio, que usa `dict[str, Any]` / dataclasses simples, não modela camada de API separada):

```
@dataclass
class FiltrosExtraidos:
    generos: list[int] = field(default_factory=list)
    sem_generos: list[int] = field(default_factory=list)
    nota_minima: float | None = None
    votos_minimos: int | None = None
    lancado_apos: str | None = None
    lancado_antes: str | None = None
    ordenar_por: str = "popularity.desc" # vira "vote_average.desc" se nota_minima foi detectada

    def tem_filtro(self) -> bool: ...
```

Refinamento sobre o plano original: quando `buscar_qualidade_no_texto` detecta um sinal de "boa avaliação", `extrair_filtros` define `ordenar_por="vote_average.desc"` em vez do padrão de

popularidade — assim "boa avaliação" não só filtra como também prioriza os melhores primeiro, mais fiel à intenção do usuário.

B) Tradução para `descobrir_filmes()`

Em `pesquisa.py`:

```
def _filtros_para_parametros_discover(f: FiltrosExtraidos) -> dict:
    return dict(
        generos=".".join(map(str, f.generos)) or None,
        sem_generos=".".join(map(str, f.sem_generos)) or None,
        nota_minima=f.nota_minima,
        votos_minimos=f.votos_minimos,
        lancado_apos=f.lancado_apos,
        lancado_antes=f.lancado_antes,
        ordenar_por=f.ordenar_por,
    )
```

Casa 1:1 com a assinatura já existente de `tmdb_filmes.descobrir_filmes()` em `backend/src/app/services/tmdb/filmes.py` — nenhuma mudança no TMDb nem no `client.py`.

C) Lógica de fallback (título vs. descoberta) — `services/pln/pesquisa.py`

```
@dataclass
class ResultadoPesquisa:
    modo: str # "título" ou "descoberta"
    resultados: list[dict]
    interpretacao: dict

def pesquisar(texto: str, pagina: int = 1, ano: int | None = None) -> ResultadoPesquisa:
    filtros = extrair_filtros(texto)
    if not filtros.tem_filtro():
        resultados = tmdb_filmes.buscar_filmes(texto, pagina=pagina, ano=ano)
        modo = "título"
    else:
        resultados = tmdb_filmes.descobrir_filmes(**_filtros_para_parametros_discover(filtros),
        modo = "descoberta"
    return ResultadoPesquisa(modo=modo, resultados=resultados, interpretacao=_montar_interpretacao)
```

Isso preserva 100% o comportamento atual para títulos comuns ("matrix", "cidade de deus" continuam indo por `buscar_filmes`) e só desvia para `/discover` quando o texto realmente carrega um sinal descritivo (gênero/qualidade/período).

D) Nova rota `GET /pesquisa`

Adicionar em `backend/src/app/api/routes/filmes.py` (não criar arquivo de rota separado — mantém o padrão de 1 router de filmes já existente):

```
from app.services.pln.pesquisa import pesquisar

@router.get("/pesquisa")
def pesquisa(
    q: str = Query(min_length=1, description="Texto livre: titulo ou descricao"),
    pagina: int = Query(1, ge=1, le=500),
    ano: int | None = Query(None, ge=1874, description="So usado se o texto virar busca por titulo")
) -> dict[str, Any]:
    try:
        resultado = pesquisar(q, pagina=pagina, ano=ano)
    except TMDbError as exc:
        raise HTTPException(status_code=502, detail=str(exc)) from exc

    return {
        "consulta": q,
        "pagina": pagina,
        "modo": resultado.modo,
        "interpretacao": resultado.interpretacao,
        "total": len(resultado.resultados),
        "resultados": resultado.resultados,
    }
```

- `/busca` continua existindo intacta (reusada internamente no modo "título"); `/pesquisa` é aditiva, não substitui nada — reduz risco.
- Mantém o nome do query param `q` (mesmo que o frontend já envia hoje), só muda o path.
- `interpretacao` é só para depuração/relatório (ex.: `{"generos_incluidos": ["Comedia"], "generos_excluidos": [], "nota_minima": 7, "votos_minimos": 100, "lancado_apos": "2016-01-01", "lancado_antes": null}`) — a UI não precisa renderizar isso, mas fica visível no Network tab / útil pra demonstrar o PLN funcionando na apresentação.
- Erro: mesmo padrão `try/except TMDbError -> HTTPException(502)` já usado nas outras rotas.

E) Mudança mínima no frontend

- `frontend/script.js`: trocar a URL chamada no submit do form de `${API_BASE}/busca?...` para `${API_BASE}/pesquisa?...` (mesmos parâmetros `q / ano / pagina`). Resto do arquivo (`renderizarResultados`, `modal`) fica idêntico — a resposta mantém o mesmo formato de `resultados`.

- `frontend/index.html` : só o `placeholder` do input muda, de "Título do filme (ex: matrix)" para algo que convide frases descritivas sem anunciar PLN — ex. "Título ou o que você quer assistir (ex: matrix, ou algo leve e bem avaliado)". Mesmo campo, mesmo `id`, mesma estrutura.

F) Testes / avaliação (sem dependência nova)

`pytest` não está instalado — respeitando a restrição de "zero dependência nova", uso `unittest` da `stdlib`:

- `backend/tests/test_extrator.py` — casos anotados manualmente (texto → filtros esperados), incluindo o exemplo original do usuário e um caso de negação:

```
CASOS = [  
    ("queria um filme descontraido com boa avaliacao que nao seja muito antigo",  
     {"generos_incluidos": ["Comedia"], "nota_minima": 7, "lancado_apos_min_ano": 2016}),  
    ("matrix", {"fallback_titulo": True}),  
    ("nao quero terror, algo emocionante dos anos 90",  
     {"generos_excluidos": ["Terror"], "generos_incluidos": ["Drama", "Romance"],  
      "lancado_apos": "1990-01-01", "lancado_antes": "1999-12-31"}),  
]
```

- Os testes populam `generos_cache` com um dict fixo pt-BR (sem bater na API TMDb real), chamam `extrair_filtros` direto.
- Função auxiliar soma acertos por "slot" (gênero incluído certo, excluído certo, nota mínima certa, período certo) e imprime uma acurácia agregada (ex. "13/15 slots corretos") — dá um número concreto pra citar no relatório do trabalho.

G) Ordem de implementação

1. `normalizacao.py` — testar manualmente com acentos/maiúsculas.
2. `lexico.py` — testar isolado (10-15 frases), sem rede.
3. `periodo.py` — testar "dos anos 90", "depois de 2015", "recente", "clássico".
4. `negacao.py` — testar "não quero terror", "sem romance".
5. `generos_cache.py` — primeiro ponto que depende de rede/token TMDb.
6. `extrator.py` — orquestra 1-5, testar com a frase completa do usuário.
7. `pesquisa.py` — testar `pesquisar("matrix")` (deve cair em modo título) e `pesquisar("comedia leve recente bem avaliada")` (deve cair em descoberta).
8. Rota `GET /pesquisa` — testar via `/docs` ou `curl`.
9. `backend/tests/test_extrator.py` — pode ser feito em paralelo aos passos 2-7.
10. Frontend (`script.js` + `index.html`) — só depois de `/pesquisa` estar estável.

Risco principal: passo 5 é o único que depende de rede/token — por isso entra só depois da lógica pura (1-4) já validada offline.

Verificação end-to-end

Mesma abordagem já usada para validar os endpoints TMDb anteriores: subir o servidor local (`uvicorn app.main:app`) e testar com `curl` :

- `curl "http://127.0.0.1:8000/pesquisa?q=queria+um+filme+descontraido+com+boa+avaliacao+que+nao+seja+muito+antigo"` → esperar `modo: "descoberta"` , `interpretacao.generos_incluidos` contendo "Comedia", `nota_minima` ~7, resultados ordenados por nota.
- `curl "http://127.0.0.1:8000/pesquisa?q=matrix"` → esperar `modo: "titulo"` , resultados equivalentes ao `/busca?q=matrix` atual.
- `curl "http://127.0.0.1:8000/pesquisa?q=nao+quero+terror,+algo+emocionante+dos+anos+90"` → esperar gênero Terror em `generos_excluidos` , período 1990-1999.
- Rodar `python -m unittest backend/tests/test_extrator.py` e conferir a acurácia por slot.
- Abrir `frontend/index.html` via `python -m http.server 5500` e confirmar visualmente que a busca continua com a mesma cara, mas frases descritivas agora trazem resultados filtrados.

Arquivos críticos

- `backend/src/app/services/pln/extrator.py`
- `backend/src/app/services/pln/pesquisa.py`
- `backend/src/app/services/pln/lexico.py`
- `backend/src/app/api/routes/filmes.py`
- `frontend/script.js`